any C++ included kernel ?

Yes, there are several production, enterprise, and hobbyist operating system kernels written partially or entirely in **C++**.

While C remains the dominant language for monolithic kernels (like Linux), C++ is highly favored in **object-oriented microkernels** and specialized embedded environments.

## 1. Major Production & Commercial Kernels

- 
- **Windows NT Kernel (Microsoft):** The core Windows kernel is predominantly C, but large parts of it—especially modern **subsystems, executive layers, and device drivers**—are written in C++.
- **Zircon (Google Fuchsia):** Google's next-generation open-source operating system uses the **Zircon microkernel**, which is explicitly written in **C++**. It utilizes C++ objects to manage system handles, tasks, and memory.
- **macOS & iOS (XNU Kernel):** While the foundational Mach microkernel layer inside XNU is written in C, the **I/O Kit** (the object-oriented device driver framework heavily integrated into the kernel) is written in a subset of **C++**.
- **Symbian OS:** The historic mobile OS that dominated the early smartphone era relied on the **EKA1 and EKA2 kernels**, which were written almost entirely from scratch in C++.
- 

## 2. Open-Source & Research Microkernels

- 
- **Genode OS Framework:** A highly secure, capability-based operating system component framework that uses **C++ as its native language**. Its base microkernels and user-space components are built tightly around C++ structures.
- **Haiku / BeOS:** Haiku, the open-source continuation of BeOS, features an architecture deeply rooted in C++. Its application frameworks, kits, and many kernel-level driver interfaces are built natively using **C++ classes**.
- **K42:** A research kernel developed by IBM and various universities designed specifically for cache-coherent multiprocessors, using a **fully object-oriented C++ design**. [1, 2, 3]
- 

---

## Why isn't every kernel written in C++?

Opinions are historically mixed among operating system architects regarding the language. [4, 5]

1. **Missing Runtime Support:** A standard kernel runs bare-metal without a user-space library. Features like C++ exceptions, Run-Time Type Information (RTTI), and standard vectors (std::vector) require a runtime system to work. Kernels written in C++ must turn

these features off or implement their own custom, bare-metal memory allocators. [4, 6]
2. **ABI Stability:** As discussed on platforms like Reddit's C++ Community, C has a highly stable Application Binary Interface (ABI). C++ functions undergo "name mangling" and binary formats can change depending on which compiler version you use, which makes maintaining external driver compatibility much harder. [5]

Are you asking because you are interested in **kernel development / OS dev** yourself, or are you exploring a specific OS architecture?


[1] https://stackoverflow.com
[2] https://iopscience.iop.org
[3] https://en.wikipedia.org
[4] https://www.reddit.com
[5] https://www.reddit.com
[6] https://wiki.osdev.org